Comparing 1-NN, 3-NN and NCC on the CIFAR-100 Image Dataset

Interim Assignment—Neural Networks and Deep Learning

***Abstract*—This report compares nearest-neighbor classification with one and three neighbors (1-NN and 3-NN) against the nearest class centroid classifier (NCC). CIFAR-100 provides 50,000 training images and 10,000 test images; each image is represented by its 3,072 RGB values. The report explains the data layout, floating-point conversion, scaling to $[0, 1]$, classification rules and accuracy measurement. The original implementation is retained as a reference and compared with a separate optimized CPU implementation. Optimization changes how the calculations are executed, without introducing new features or reducing the training set. Full-test accuracies are 17.55%, 17.48% and 10.04%, respectively. The shared neighbor search for both k-NN classifiers takes 10.32 seconds in the measured CPU run.**

***Index Terms*—CIFAR-100, image classification, k-NN, nearest class centroid, Euclidean distance, accuracy, computational cost.**

I. Objective and Experimental Approach

The objective is to compare three simple classification algorithms on the same dataset. The first question is how accurately each rule classifies unseen images when raw pixel values are used as features. The second is how the computational cost differs between retaining all training examples, as in k-NN, and representing each class by a single centroid, as in NCC.

This interim assignment does not train a neural network. No PCA, data augmentation or pretrained feature extractor is applied. The comparison therefore concerns these three classifiers in the same RGB feature space. The values $k = 1$ and $k = 3$ are specified by the assignment rather than selected by tuning on test results [3].

II. Dataset and Representation

CIFAR-100 contains color images of 32×32 pixels from 100 classes. Each class has 500 training and 100 test images. The classes include animals, plants, objects, vehicles and scenes. This experiment uses the `fine_labels`, not the 20 superclass labels [1], [2].

The Python distribution provides `train`, `test` and `meta` files. The first two hold images and labels; the last maps numeric class IDs to class names. The official split is retained: test images are neither included among candidate neighbors nor used to calculate class centroids.

A. From an Image to a Matrix Row

Each image has three color channels, so its feature count is

$$D = 32 \cdot 32 \cdot 3 = 3\,072. \tag{1}$$

The stored images are already flattened: the first 1,024 entries are the red channel, followed by 1,024 green and 1,024 blue values. A row of X is one complete image; a column corresponds to a particular pixel location and channel. The label y_i describes the image in row i. In NumPy, y is a one-dimensional array, not a 100-column matrix or a one-hot encoding.

Table I
Input arrays and class labels

Variable	NumPy shape	Contents
X_{train}	$(50\,000, 3\,072)$	Training images
y_{train}	$(50\,000,)$	Training class IDs
X_{test}	$(10\,000, 3\,072)$	Test images
y_{test}	$(10\,000,)$	True test class IDs

For display, a row is reshaped to $(3, 32, 32)$ and then transposed to $(32, 32, 3)$, as required by `imshow`. This rearrangement is only for visualization and is not part of classification.

B. Preprocessing

The original values are stored as `uint8` in $[0, 255]$. Before numerical operations, they are converted to `float32` and divided by 255:

$$\widetilde{x}_{ij} = \frac{\text{float32}(x_{ij})}{255}. \tag{2}$$

Labels remain integers. Conversion prevents unsigned-integer wraparound during subtraction and allows fractional values. Division changes the scale: for example, $0 \mapsto 0$, $51 \mapsto 0.2$ and $255 \mapsto 1$.

The same function processes both splits. No scaling parameters are learned from the test set: 255 is the known maximum of the original representation. In exact arithmetic, this common scaling preserves Euclidean distance order:

$$d(\mathbf{x}/255, \mathbf{z}/255) = d(\mathbf{x}, \mathbf{z})/255. \tag{3}$$

Scaling alone is therefore not expected to improve k-NN or NCC accuracy.

C. Grouping by Class

The original implementation constructs a dictionary, `points`, whose keys are class IDs and whose values contain the corresponding training images:

$$\texttt{points}[c] = X_{\text{train}}[y_{\text{train}} = c], \quad \text{shape } (500, 3\,072). \tag{4}$$

The expression $y = c$ creates a Boolean mask that selects rows with the corresponding label. Only training images enter this grouping.

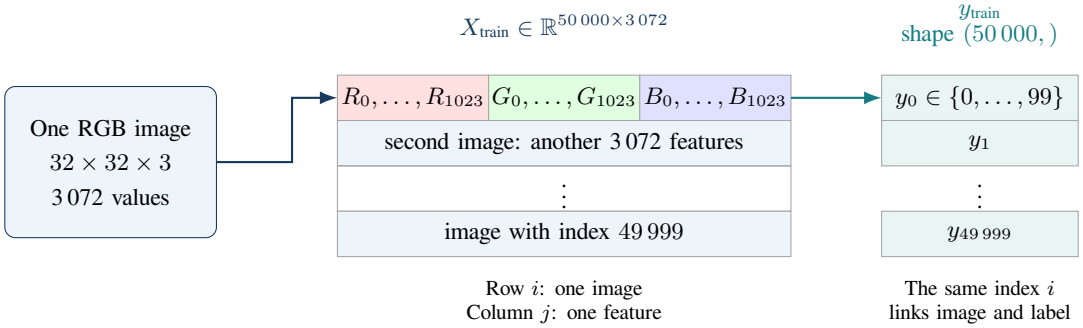


Figure 1. Images and their labels. Flattening changes the layout, not the pixel values. The test set has the same structure with 10 000 rows.

III. The Three Classifiers

A. Euclidean Distance

The shared distance measure between two feature vectors is

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{D} (x_j - z_j)^2}. \tag{5}$$

Every feature has equal weight. There is no special treatment of neighboring pixels, contours or object shapes. Images may be close because of similar backgrounds or colors, even when they depict different kinds of objects.

B. 1-NN and 3-NN

For k-NN, `fit()` stores the training examples. It does not learn weights or iteratively minimize an error function. Given a new image, distances to all training images are calculated and the k nearest examples form the neighborhood $\mathcal{N}_k(\mathbf{x})$. The prediction is

$$\widehat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]. \tag{6}$$

The 1-NN rule copies the nearest example's class. The 3-NN rule uses a majority vote among three examples. Votes are not distance-weighted.

Two kinds of ties need a rule. First, when training images have exactly the same distance from the test image, the image with the smaller numerical class ID is placed first. This gives a consistent ordering when selecting the nearest neighbors; the class ID does not measure similarity.

Second, the three selected neighbors may all belong to different classes. For example, labels A, A, B give class A two votes, so A wins. Labels A, B, C give each class one vote. In this implementation, `Counter` breaks that voting tie by choosing the label that appears first in the sorted neighbor list. Therefore, the closest neighbor's class wins; if distances also tie, the smaller-class-ID rule above determines which comes first.

The sorting key contains only distance and class ID. The stored image arrays are excluded because NumPy compares arrays element by element, producing multiple Boolean values rather than the single true-or-false decision Python needs to order two entries.

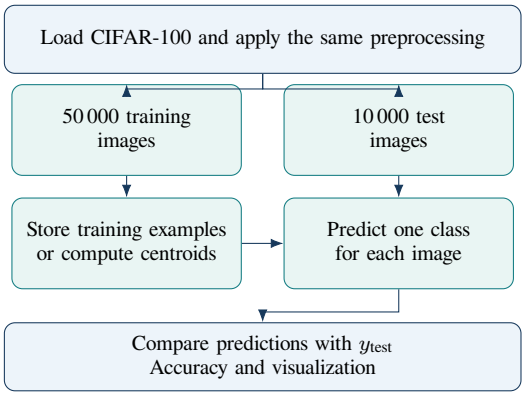


Figure 2. Training and evaluation are separate stages. True test labels are used to score predictions, not to fit the classifiers.

C. Nearest Class Centroid (NCC)

For each class c, the centroid is

$$\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i:y_i=c} \mathbf{x}_i, \qquad n_c = 500. \tag{7}$$

The operation `mean(axis=0)` averages each column, hence each pixel position and channel, across the class's training images. This yields 100 vectors of length 3,072. A new image is assigned to

$$\widehat{y} = \arg\min_{c \in \{0,\ldots,99\}} d(\mathbf{x}, \boldsymbol{\mu}_c). \tag{8}$$

NCC compares an image with 100 centroids instead of 50,000 examples. This saves computation but summarizes potentially diverse appearances of a class by one mean vector. Exact distance ties select the first minimum; classes are in ascending order in the experiment.

IV. Code Organization and Optimization

A. Original Implementation

The file `src/main.py` separates responsibilities into functions:

- `load_dataset`: read the data files and class names.
- `preprocess_data`: convert and scale images; extract labels.
- `group_by_class`: group training examples by class.
- `fit_classifiers`: create and fit the three models.
- `evaluate_classifier`: collect predictions and calculate accuracy.

- `plot_predictions`: show images with true and predicted labels.

The `main()` function connects these stages. Evaluation finishes before plot windows open, and visualization reuses saved predictions rather than classifying the displayed images again.

The original k-NN search is easy to follow, but performs a Python loop over every training image and sorts every distance. A full test pass requires

$$50\,000 \cdot 10\,000 = 500\,000\,000 \tag{9}$$

image-pair comparisons, each involving 3,072 features. Separate runs for $k = 1$ and $k = 3$ duplicate the distance calculations. This implementation works; its high runtime motivated optimization rather than indicating an algorithmic failure.

B. Separate Optimized Implementation

The file `src/optimized.py` introduces `OptimizedKNearestNeighbors` and `OptimizedNearestClassCentroid`, while retaining the original implementation separately. All training images remain candidates; the search is exhaustive, not approximate.

For a batch of B test images, squared Euclidean distances use the identity

$$\|\mathbf{x} - \mathbf{z}\|_2^2 = \|\mathbf{x}\|_2^2 + \|\mathbf{z}\|_2^2 - 2\mathbf{x}^\mathsf{T}\mathbf{z}. \tag{10}$$

This is the vector equivalent of the familiar scalar expansion $(a - b)^2 = a^2 + b^2 - 2ab$. Here, $\|\mathbf{x}\|_2^2$ is the sum of the squared components of $\mathbf{x}$, and $\mathbf{x}^\mathsf{T}\mathbf{z}$ is the dot product, the sum of products of corresponding components. The left-hand side is the squared distance, rather than the distance itself.

This form lets us calculate distances for a whole batch using efficient BLAS matrix multiplication to compute the dot products. Squared training norms are calculated once during `fit()` and reused for every batch. We can also omit the square root because squared distances give the same nearest-neighbor ordering: taking a square root preserves the order of nonnegative values.

With batches of $B = 128$, the distance matrix has shape $128 \times 50\,000$ and occupies about 51.2 MB in `float64`. The code does not construct a three-dimensional difference array of shape $B \times 50\,000 \times 3\,072$, or retain distances for all 10,000 test images at once. Batching controls memory; array operations and less work inside Python loops provide the speed improvement.

Instead of sorting all 50,000 values, `partition` finds the distance of the kth nearest example. Only candidates at or below that threshold are sorted, including all ties at the boundary. Their final order uses distance, class ID and training-example order. The nearest three neighbors are sufficient to produce both 1-NN and 3-NN predictions from one search.

The optimized NCC retains the original class-mean calculation and uses the same distance identity against the 100 centroids. Its class order on exact distance ties is unchanged.

Inputs are normalized `float32` values, while optimized distance arithmetic uses `float64` to reduce cancellation error in (10). Tiny negative computed squared distances are clipped to zero. The mathematical distance is unchanged, but bitwise equivalence with the original arithmetic is not guaranteed; nearly tied distances require care. Predictions are therefore explicitly compared against the original implementation.

Table II
FULL EVALUATION ON 10,000 TEST IMAGES

Classifier	Correct predictions	Accuracy
1-NN	1 755	17.55%
3-NN	1 748	17.48%
NCC	1 004	10.04%

Table III
PREDICTION TIME ON THE SAME 100-IMAGE SAMPLE

Model	Original (s)	Optimized (s)	Speedup
1-NN	24.173	0.0916	263.8×
3-NN	24.031	0.1095	219.5×
NCC	0.0417	0.000589	70.9×

V. Evaluation and Results

A. Protocol and Metric

All classifiers use the same input representation and labels. Accuracy is

$$A = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1}[\widehat{y}_i = y_i]. \tag{11}$$

It measures the fraction of correctly classified images, and differs from per-class precision. The test set is balanced over 100 classes, making accuracy a direct basis for comparison. Always predicting a fixed class would achieve exactly 1% on the full test set.

Prediction timing excludes data loading and plotting. The preliminary 20-image run was only a functionality and runtime check, not a substitute for the final comparison. Speed comparisons use the same test examples and training set. Extrapolated runtimes are identified as estimates rather than measurements of full runs.

B. Measurements

Measurements used an AMD Ryzen 9 7950X CPU, eight OpenBLAS threads, Python 3.10.12, NumPy 1.24.3 and OpenBLAS 0.3.21. Batch size was 128. No GPU was used. Each reported prediction time is one wall-clock measurement after a short warm-up, not an average of repeated runs. Timings depend on the machine and its concurrent workload.

The common comparison sample contained one test image per class, selected with `default_rng` and seed 2026: 100 images in total. Every classifier used all 50,000 training examples. Each optimized model agreed with its original on **all 100 predictions**. This validates the sample, not every possible numerical tie. The original NCC was also run on the entire test set, yielding agreement on all 10,000 predictions.

Table III uses separate prediction calls for each classifier. In the final full run, the shared 1-NN/3-NN search took **10.319 s** for all 10,000 images. This is the time for *both results*

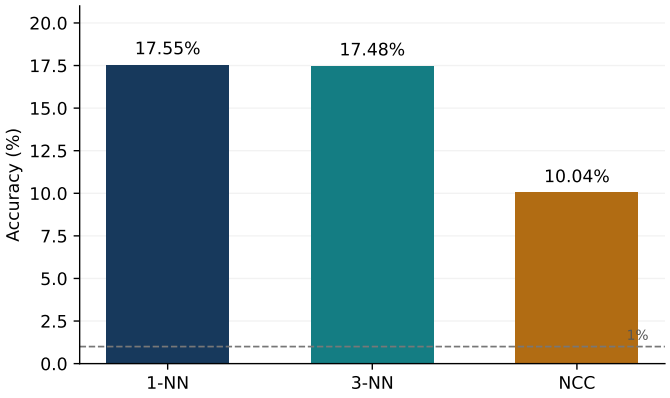


Figure 3. Measured accuracy of the optimized implementations on the full test set. The dashed line marks the 1% achieved by a constant class prediction on this balanced dataset.

together, not an independent runtime for each model. It includes neighbor selection, voting and progress messages. Optimized NCC prediction took 0.0567 s; the original NCC took 4.158 s on the same full set.

The original k-NN `fit()` only stores a dictionary reference and was measured at approximately 1 μs. Shared optimized k-NN preparation took 1.523 s because it constructs a contiguous `float64` matrix, labels and squared norms. This preparation is paid once and excludes loading and grouping. NCC centroid construction took about 0.023 s in both versions.

The original k-NN implementations were not run on the entire test set. Extrapolating their sample timings suggests about 40 minutes per original pass; this is an *estimate*, not a measured full run. The original implementation is functional. The observed CPU acceleration made GPU support and PCA unnecessary for this experiment.

The files `results/cpu_benchmark.json` and `results/predictions.npz` retain the settings, measurements and full predictions in original test-set order. The JSON also records sample indices and SHA-256 hashes of the measured source files.

C. Classification Examples

Figure 4 shows actual predictions from the full run. For each classifier, the first two correct and first two incorrect predictions are selected in test-index order. This selection is reproducible, but is not a random or representative sample of overall performance. Class names retain the dataset's original spelling.

VI. Discussion and Limitations

Euclidean distance on raw RGB values is not invariant to translation, orientation or lighting changes. Low image resolution also limits the available information. An object that is easily recognized by a person need not be close to images of its own class in this feature space.

Using three neighbors can reduce the influence of an individual example, but does not guarantee higher accuracy than 1-NN: the extra neighbors may belong to another class. NCC substantially reduces prediction work, but a single centroid cannot represent multiple appearance clusters within a class. The measured ranking concerns this dataset, representation and distance measure, not a universal ordering of the algorithms.

This benchmark dataset does not establish performance on other image sources or real-world applications. No independent image-by-image audit of similarities between training and test images was performed. The official split was retained, and no personal image collections were added. Availability for download does not by itself establish unrestricted reuse rights for the underlying images.

VII. Conclusion

In this experiment, 1-NN achieved 17.55%, 3-NN 17.48% and NCC 10.04%. The k-NN results are very close: the difference is only seven correct predictions, or 0.07 percentage points. Without further statistical analysis, this does not establish general superiority of 1-NN. Both k-NN models outperformed NCC in this feature space, while NCC had much lower prediction cost.

Distinguishing the algorithm from its implementation was important. The original code made the classification rules explicit; the optimized version used matrix operations, partial neighbor selection and a shared search for both values of k. Agreement with the reference was checked before using the final results. The full shared k-NN search completed in about 10.3 s on CPU, without reducing the training set or applying PCA. The modest accuracy illustrates that faster execution does not add semantic information to raw RGB features.

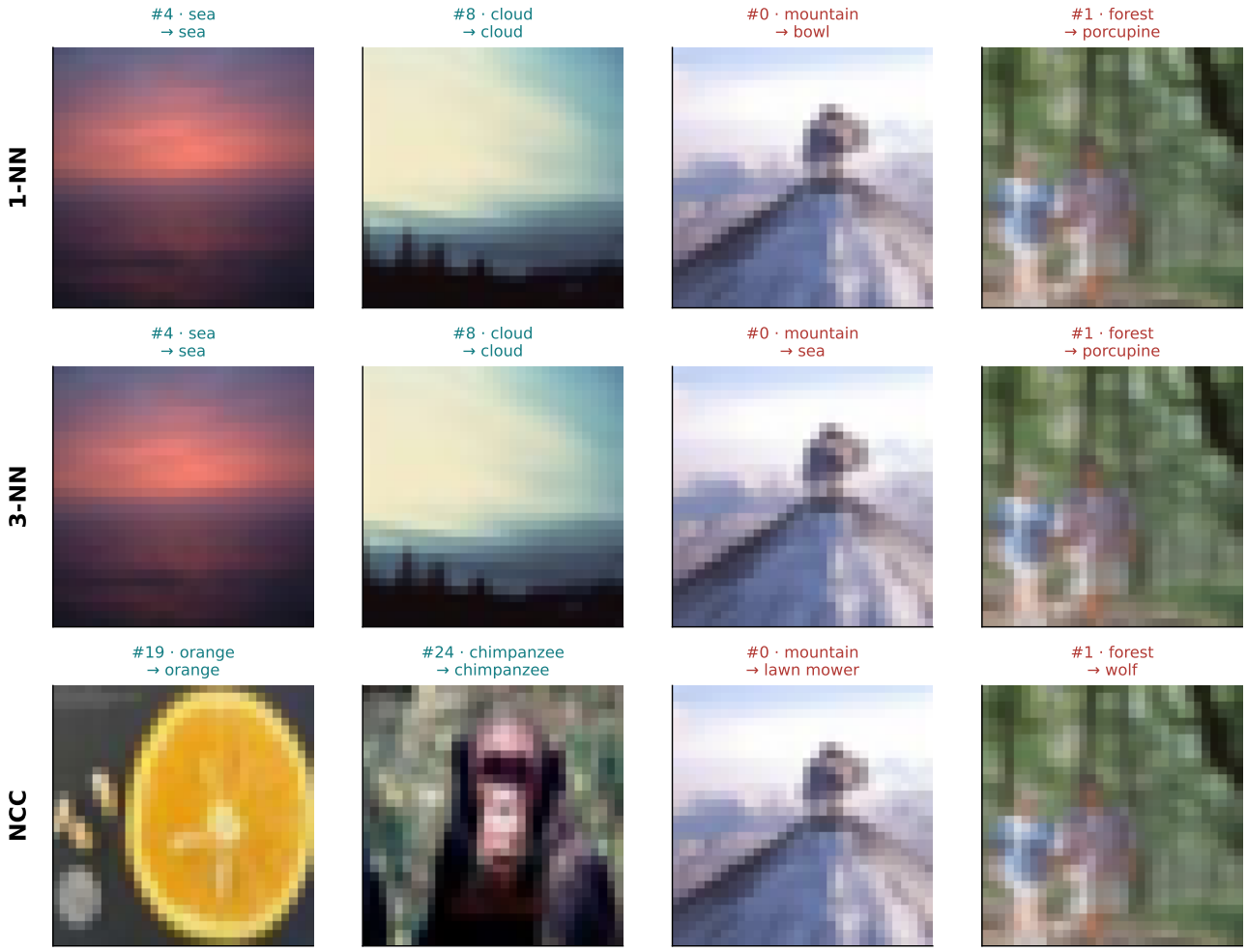


Figure 4. One row per classifier. The two left columns contain correct predictions (green titles), and the two right columns contain errors (red titles). Above each image are its test index and true class; the predicted class follows the arrow.

Appendix A
Original Implementation: Key Excerpts

These excerpts are included directly from `src/main.py`. The complete accompanying file also contains data loading, preprocessing, execution and plotting.

```python
def euclidean_distance(p, q):
    return np.sqrt(np.sum((np.array(p) - np.array(q)) ** 2))
```

```python
    def k_nearest(self, new_point):
        """Return the k closest training points as (distance, category, point)."""
        distances = []

        for category, class_points in self.points.items():
            for point in class_points:
                distance = euclidean_distance(point, new_point)
                distances.append((distance, category, point))

        # Exclude image arrays from comparisons when distance and category tie.
        return sorted(distances, key=lambda item: (item[0], item[1]))[: self.k]

    def predict(self, new_point):
        categories = [category for _, category, _ in self.k_nearest(new_point)]
        return Counter(categories).most_common(1)[0][0]
```

```python
    def fit(self, points):
        # Example for this dataset: {"blue": [2.0, 2.6], "red": [4.8, 5.4]}
        self.centroids = {
            category: np.mean(class_points, axis=0)
            for category, class_points in points.items()
        }

    def predict(self, new_point):
        return min(
            self.centroids,
            key=lambda category: euclidean_distance(
                self.centroids[category], new_point
            ),
        )
```

```python
def evaluate_classifier(classifier, X_test, y_test):
    """Return predictions and accuracy for an already fitted classifier."""
    if len(X_test) == 0:
        raise ValueError("The evaluation set must contain at least one image.")
    if len(X_test) != len(y_test):
        raise ValueError("Each evaluation image must have one label.")

    predictions = []
    correct_predictions = 0

    for i in range(len(X_test)):
        prediction = classifier.predict(X_test[i])
        predictions.append(prediction)
        if prediction == y_test[i]:
            correct_predictions += 1

    accuracy = correct_predictions / len(X_test)
    return np.array(predictions), accuracy
```

Appendix B
Optimized Implementation: Key Excerpts

These excerpts are taken from the accompanying `src/optimized.py`. The shared distance function is used for both training examples and class centroids.

```python
def squared_distances(queries, references, reference_norms):
    """Use ||x-y||² = ||x||² + ||y||² - 2 x·y without a 3D temporary."""
    distances = queries @ references.T
    distances *= -2.0
    distances += np.einsum("ij,ij->i", queries, queries)[:, None]
    distances += reference_norms[None, :]
    # Cancellation can produce tiny negative values for coincident points.
    np.maximum(distances, 0.0, out=distances)
    return distances
```

```python
        for start in range(0, len(X), self.batch_size):
            stop = min(start + self.batch_size, len(X))
            distances = squared_distances(X[start:stop], self.X, self.norms)
            for offset, row in enumerate(distances):
                threshold = np.partition(row, k - 1)[k - 1]
                # Include all boundary ties before ordering by distance/category/index.
                candidates = np.flatnonzero(row <= threshold)
                order = np.lexsort((candidates, self.labels[candidates], row[candidates]))
                neighbors[start + offset] = candidates[order[:k]]
            if progress:
                print(f"k-NN search: {stop}/{len(X)} images", flush=True)
        return neighbors
```

```python
    def predict_for_k(self, X, ks=(1, 3), progress=False):
        """Reuse one exhaustive search for several values of k."""
        if not ks or any(not isinstance(k, int) or k < 1 for k in ks):
            raise ValueError("Provide positive integer neighbor counts.")
        indices = self.kneighbors(X, k=max(ks), progress=progress)
        labels = self.labels[indices]
        return {
            k: np.array([
                Counter(row[:k]).most_common(1)[0][0] for row in labels
            ], dtype=self.labels.dtype)
            for k in ks
        }
```

Optimized NCC compares each batch against the class centroids using the same shared distance function:

```python
    def predict_many(self, X):
        if self.centroids is None:
            raise ValueError("Fit the classifier before predicting.")
        X = query_array(X, self.centroids.shape[1])
        predictions = np.empty(len(X), dtype=self.labels.dtype)
        for start in range(0, len(X), self.batch_size):
            stop = min(start + self.batch_size, len(X))
            distances = squared_distances(X[start:stop], self.centroids, self.norms)
            predictions[start:stop] = self.labels[np.argmin(distances, axis=1)]
        return predictions
```

The file `src/benchmark.py` performs agreement checks, timing and full evaluation. Complete source files accompany this report.

References

[1] A. Krizhevsky, V. Nair and G. Hinton, “CIFAR-10 and CIFAR-100 datasets.” https://www.cs.toronto.edu/~kriz/cifar.html.
[2] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, technical report, University of Toronto, 2009, ch. 3. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.
[3] First compulsory assignment, “Neural Networks—Deep Learning,” section “Interim Assignment,” 2026.